

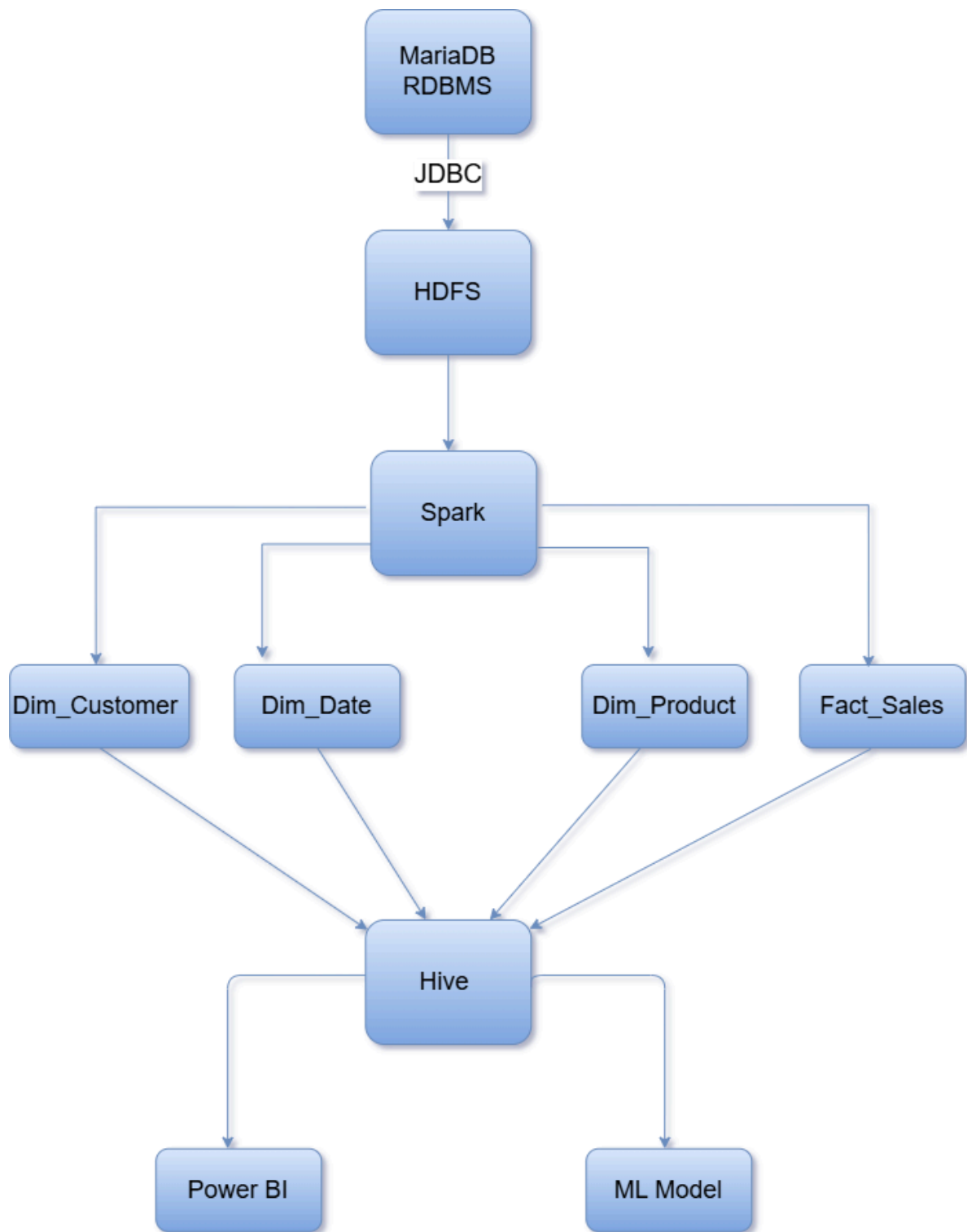
1. Project Overview & Business Goal

This project focuses on building and managing two essential dimension tables for an e-commerce data warehouse: `Dim_Customer` and `Dim_Product`.

The main focus is on analyzing customer behavior using RFM (Recency, Frequency, and Monetary) analysis and implementing logic for historical product price tracking.

The project transforms customer and product data into structured dimensional tables that support customer segmentation and historical product analysis. The resulting dimensional model provides a foundation for understanding customer behavior and product performance and supports data-driven business analysis.

2. Architecture & Technologies



3. Data Ingestion

Raw data for this project originates from the Olist Brazilian E-Commerce dataset (Kaggle). Five source tables — customers, products, orders, order items, and order payments — were first loaded into a MariaDB relational database, then ingested into Apache Spark for processing.

3.1 Batch Load — CSV to MariaDB

A MySQL/MariaDB database (olist_raw) was created with a table for each raw dataset. Data was loaded using LOAD DATA LOCAL INFILE, which reads each CSV file directly into its corresponding table. This step corresponds to the "Batch Load" ingestion method referenced in the project guidelines, as an alternative to Sqoop given the moderate data volume (~99,000 rows).

3.2 MySQL to Spark — JDBC Connection

Apache Spark connects to the MariaDB instance through a JDBC driver. Each raw table is read into a Spark DataFrame, making the data available for distributed processing.

```
In [4]: def read_table(table_name: str):  
        return spark.read.jdbc(url=JDBC_URL, table=table_name, properties=JDBC_PROPERTIES)
```

```
In [9]: raw_products = read_table("raw_products")  
raw_orders = read_table("raw_orders")  
raw_order_items = read_table("raw_order_items")  
raw_order_payments = read_table("raw_order_payments")  
  
print("")  
for name, df in [  
    ("customers", raw_customers),  
    ("products", raw_products),  
    ("orders", raw_orders),  
    ("order_items", raw_order_items),  
    ("order_payments", raw_order_payments),  
]:  
    print(f"{name}: {df.count()}")
```

```
customers: 99441  
products: 32951  
orders: 99441  
order_items: 112650  
order_payments: 103886
```

3.3 Landing Raw Data on HDFS (Raw Zone)

Before any cleaning takes place, the raw DataFrames are written to HDFS as Parquet files under a dedicated Raw Zone path. This mirrors the standard Data Lake pattern — data is landed exactly as received, before any transformation — and gives the pipeline a recoverable checkpoint independent of the source database.

4. Data Transformation

The data transformation stage prepares the ingested data for analytical use. The process includes data cleaning, historical product price tracking, product dimensional modeling, and customer-focused

transformations using RFM analysis and customer segmentation.

4.1 Data Cleaning

Each raw table is validated and cleaned independently before being used downstream:

- Duplicate removal on each table's primary key (dropDuplicates)
- Rejection of rows with missing critical identifiers (customer_id, product_id, order_id)
- Standardization of text fields — trimming whitespace and normalizing case for city, state, and category names
- Replacing missing product categories with "unknown" instead of dropping the row
- Rejecting invalid numeric values — non-positive prices, payment values, and product dimensions
- Handling MySQL zero-date artifacts (0000-00-00) via the zeroDateTimeBehavior=convertToNull JDBC option

The cleaned DataFrames (customers_clean, products_clean, orders_clean, order_items_clean, order_payments_clean) are exported as Parquet files for use by the rest of the team.

order_id	payment_sequential	payment_type	payment_installments	payment_value	duplicate_count
2e2c60b99754ae1e4d8b18846cfec9f2	1	credit_card	4	542.66	2
881206af1a6a9cd34119b99782759dd4	1	boleto	1	44.77	2
04f1827088d972a62224f5203a071500	1	credit_card	10	178.06	2
6ed3f553337457d7804e8c39b580ce89	1	credit_card	10	113.62	2

4.2 Historical Price Tracking

Product price is not stored as a static attribute — it is derived from every recorded sale of that product in order_items, joined with the purchase date from orders. This produces a full price history per product rather than a single current price.

The transformation logic proceeds in four steps:

- Join order_items with orders to attach a purchase date to every recorded price
- Remove same-day duplicate price entries for the same product

- Use a Window function (LAG) partitioned by product to detect the exact date each price actually changed
- Use a Window function (LEAD) to close each price period — the day before the next price change, or 9999-12-31 if the price is still current

4.3 Dim_Product

The final Dim_Product table combines the price-history records with the static product attributes (category, weight, dimensions) from products_clean, and assigns a surrogate key to each row.

product_key	product_id	product_category_name	product_weight_g	product_length_cm	product_height_cm	product_width_cm	product_price	dw_start_date	dw_end_date
8589934592	00066f42aeeb9f3007548bb9d3f33c38	perfumaria	300	20	16	16	101.65	2018-05-20	9999-12-31
541165879296	00088930e925c41fd95ebfe695fd2655	automotivo	1225	55	10	26	129.90	2017-12-12	9999-12-31
1357209665536	0009406fd7479715e4bef61dd91f2462	cama_mesa_banho	300	45	15	35	229.00	2017-12-21	9999-12-31
953482739712	000b8f95fcb9e0096488278317764d19	utilidades_domesticas	550	19	24	12	58.90	2018-08-01	9999-12-31
1142461300736	000d9be29b5207b54e86aa1b1ac54872	relogios_presentes	250	22	11	15	199.00	2018-04-03	9999-12-31
317827579904	0011c512eb256aa0dbbb544d8dffcf6e	automotivo	100	16	15	16	52.00	2017-12-14	9999-12-31
60129542144	00126f27c813603687e6ce486d909d01	cool_stuff	700	25	5	15	249.00	2017-09-17	9999-12-31
34359738368	001795ec6f1b187d37335e1c4704762e	consoles_games	600	30	20	20	38.90	2017-10-28	9999-12-31
249108103168	001b237c0e9bb435f2e54071129237e9	cama_mesa_banho	6000	40	4	30	78.90	2018-08-09	9999-12-31
1632087572480	001b72dfd63e9833e8c02742adf472e3	moveis_decoracao	600	26	8	22	34.99	2017-02-15	9999-12-31

4.4 RFM Analysis

RFM analysis was applied to evaluate customer purchasing behavior using three metrics: **Recency**, **Frequency**, and **Monetary**.

The analysis uses customer_unique_id as the customer identifier to represent the actual customer. Canceled orders were excluded from the analysis, and only valid customer orders were considered.

Recency

Recency measures how recently a customer made a purchase. It was calculated as the number of days between the latest valid purchase date in the dataset and the customer's most recent purchase date. A lower Recency value indicates a more recent purchase.

Frequency

Frequency represents the number of distinct valid orders placed by each customer.

Monetary

Monetary represents the customer's total spend. It was calculated using payment_value from the cleaned payment data. Payment values were first aggregated at the order level and then summed for each customer.

After calculating the three RFM metrics, each metric was assigned a score from 1 to 5 using **NTILE(5)**.

For Recency, lower values are better, so the scoring direction was reversed. For Frequency and Monetary, higher values receive higher scores.

customer_unique_id	recency	frequency	monetary	recency_score	frequency_score	monetary_score
317cfc692e3f86c45...	9	1	9.59	5	1	1
bd06ce0e06ad77a7f...	355	1	10.07	2	4	1
b33336f46234b24a6...	74	1	10.89	5	1	1
2878e5b88167faab1...	309	1	11.63	2	4	1
6f5b9d1cdccc4d28f...	366	1	11.63	2	4	1
809ca96e9696b9be5...	464	1	12.28	1	5	1
7859a40482024a3d0...	95	1	12.39	5	1	1
a2cab25fa1a8a53ba...	103	1	12.89	4	1	1
defbae0ddd4aeca56...	448	1	12.89	1	5	1
3ee6e9979973bf7bf...	104	1	13.29	4	2	1

The resulting RFM_SCORE is a three-digit code formed by concatenating the Recency, Frequency, and Monetary scores. For example, an RFM score of 424 represents a Recency score of 4, a Frequency score of 2, and a Monetary score of 4.

customer_unique_id	Segment	recency_score	frequency_score	monetary_score	RFM_SCORE
317cfc692e3f86c45c95697c61c853a6	New Customers	5	1	1	511
bd06ce0e06ad77a7f681f1a4960a3cc6	At Risk	2	4	1	241
b33336f46234b24a613ad9064d13106d	New Customers	5	1	1	511
2878e5b88167faab17d4fb83a986d38b	At Risk	2	4	1	241
6f5b9d1cdccc4d28f0483a612edecacf	At Risk	2	4	1	241
809ca96e9696b9be5f69cd7ae803049d	Others	1	5	1	151
7859a40482024a3d00041c4ca1434298	New Customers	5	1	1	511
a2cab25fa1a8a53ba04f8e7a78594b32	Others	4	1	1	411
defbae0ddd4aeca563944124fbcc4bae	Others	1	5	1	151
3ee6e9979973bf7bf53518c4978cbae1	Potential Loyalists	4	2	1	421
062d40ed42ac84fe9d779277fb7ed109	Potential Loyalists	4	2	1	421

4.5 Customer Segmentation

Based on the RFM scores, customers were grouped into different behavioral segments. The segmentation rules use the Recency and Frequency scores to classify customers according to their purchasing behavior.

4.6 Dim_Customer

The Dim_Customer table was created by combining customer information with the calculated RFM metrics, RFM scores, and customer segments.

A surrogate key, cust_key, was generated for the customer dimension. Customer location attributes such as customer_city and customer_state were also included to support geographical analysis, Also The cust_key is used to establish the relationship between Dim_customer and Fact_Sales.

+-----+-----+-----+-----+-----+-----+					
+-----+	+-----+-----+-----+-----+-----+				
	cust_key	customer_unique_id	customer_city	customer_state	RFM_SCORE Segment
	+-----+-----+-----+-----+-----+				
+-----+	+-----+-----+-----+-----+-----+				
1	0000366f3b9a7992bf8c76cfd3221e2	cajamar	SP	424	Potential Loyalist
s	0000b849f77a49e4a4ce2b2a4ca5be3f	osasco	SP	421	Potential Loyalist
2	0000f46a3911fa3c0805444483337064	sao jose	SC	152	Others
3	0000f6ccb0745a6a4b88665a16c9f078	belem	PA	241	At Risk
4	0004aac84e0df4da2b147fca70cf8255	sorocaba	SP	244	At Risk
5	0004bd2a26a76fe21f786e4fbd80607f	sao paulo	SP	424	Potential Loyalist
s	00050ab1314c0e55a6ca13cf7181fecf	campinas	SP	421	Potential Loyalist
7	00053a61a98854899e70ed204dd4baf	curitiba	PR	325	Others
8	0005e1862207bf6ccc02e4228effd9a0	teresopolis	RJ	154	Others
9	0005ef4cd20d2893f0d9fbd94d3c0d97	sao luis	MA	423	Potential Loyalist
10	0006fdc98a402fceb4eb0ee528f6a8d4	mimoso do sul	ES	151	Others
s	00082cbe03e478190aadbea78542e933	itapeva	SP	243	At Risk
11					
12					

4.6 Dim_Date

The Dim_Date table was created from the order purchase dates and contains the main calendar attributes required for time-based analysis.

The dimension includes the following attributes:

- date_key – unique key used to link dates with the fact table.
- full_date – the complete calendar date.
- year – year of the date.
- quarter – quarter of the year.
- month – month of the year.
- day – day of the month.

The date_key is used to establish the relationship between Dim_Date and Fact_Sales, enabling time-based analysis of orders and sales.

4.7 Fact_Sales

The Fact_Sales table was generated by combining the required transactional data from orders, products, and payments. It contains the main sales and order-related information required for analysis.

The fact table includes:

- sales_key – unique key for each sales record.
- order_id – stored directly in the fact table as a degenerate dimension.
- cust_key – foreign key referencing Dim_Customer.
- product_key – foreign key referencing Dim_Product.
- order_purchase_date_key – foreign key referencing Dim_Date.
- order_status – status of the order.
- price – product price.
- freight_value – shipping/freight value.
- payment_value – payment amount.
- payment_type – payment method.
- order_year and order_month – partitioning attributes used to organize the fact data.

The resulting Fact_Sales table serves as the central table of the Star Schema and connects the customer, product, and date dimensions.

5. Data Warehouse Insights

5.1 Hive

Apache Hive was used as the data warehouse layer to organize and query the transformed e-commerce data.

The dimensional and fact tables were created as external Hive tables and stored using the Parquet file format. The warehouse contains the following main tables:

Dim_Customer

Dim_Product

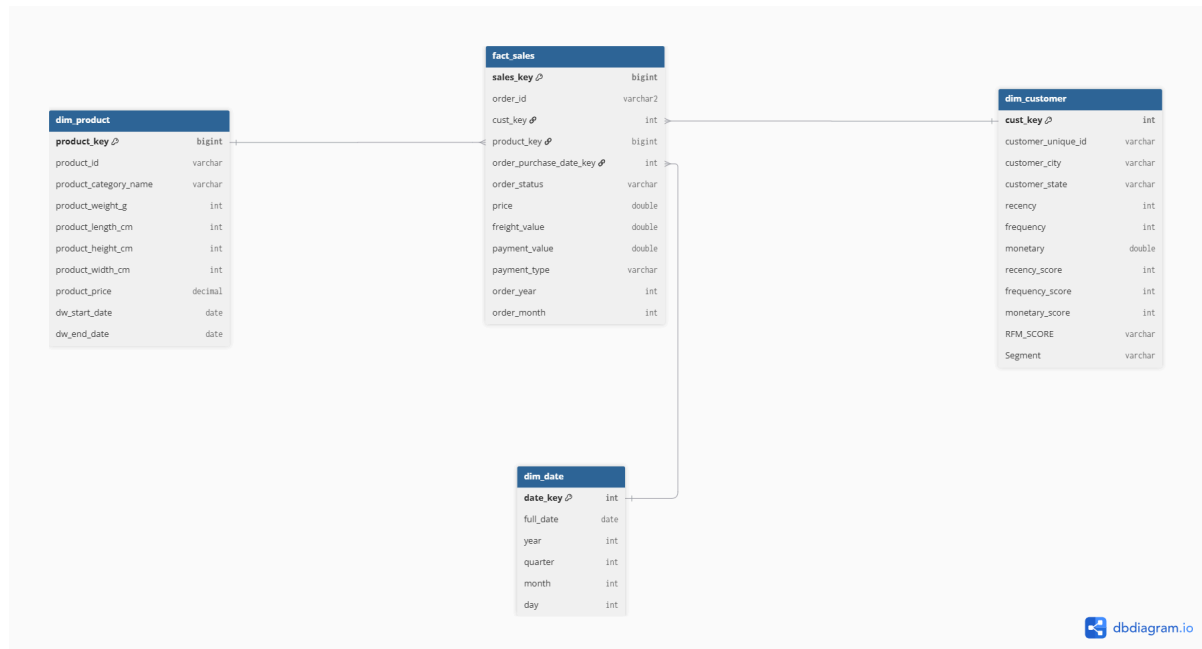
Dim_Date

Fact_Sales

Our tables were organized to support analytical queries and provide a structured foundation for business intelligence and reporting.

5.2 Star Schema

The data warehouse follows a Star Schema design, with Fact_Sales serving as the central fact table and the dimension tables surrounding it.



5.4 Hive Queries & Data Warehouse Insights

The following are some of the key analytical queries used to extract insights from the data warehouse:

```
-- Top 5 States by Customers
SELECT
    customer_state,
    COUNT(DISTINCT customer_unique_id) AS total_customers
FROM dw.dim_customer
GROUP BY customer_state
ORDER BY total_customers DESC
LIMIT 5;
```

```
-- Orders Trend Over Time - Quarterly (Using dim_date)
SELECT
    CONCAT(d.year, '-Q', d.quarter) AS quarter_year,
    COUNT(DISTINCT f.order_id) AS total_orders
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.order_purchase_date_key = d.date_key
GROUP BY d.year, d.quarter
ORDER BY quarter_year;
```

```
-- Top 5 Product Categories
SELECT
    p.product_category_name,
    COUNT(f.sales_key) AS total_products_sold
FROM dw.fact_sales f
JOIN dw.dim_product p ON f.product_id = p.product_id
WHERE p.product_category_name IS NOT NULL
GROUP BY p.product_category_name
ORDER BY total_products_sold DESC
LIMIT 5;
```

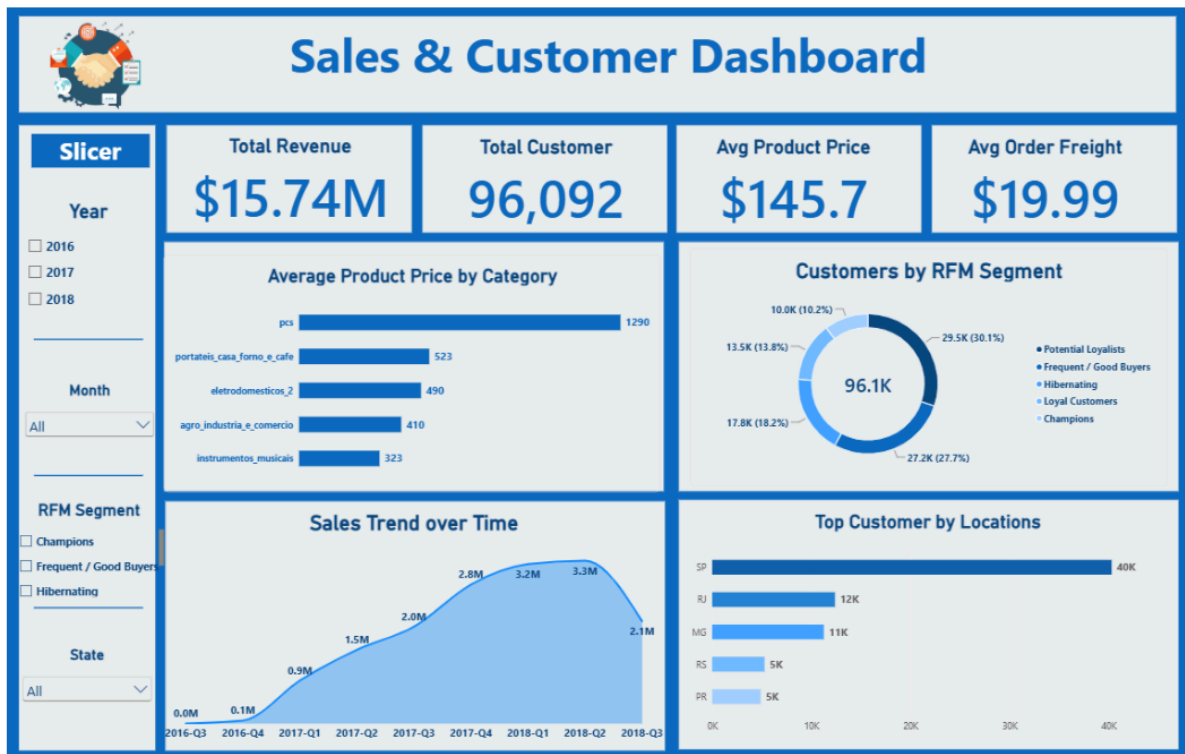


```
-- Orders Status Breakdown
SELECT
    order_status,
    COUNT(DISTINCT order_id) AS total_orders
FROM dw.fact_sales
GROUP BY order_status
ORDER BY total_orders DESC;
```

```
-- Busiest Purchase Hours (Extracted from full_date)
SELECT
    HOUR(d.full_date) AS purchase_hour,
    COUNT(DISTINCT f.order_id) AS total_orders
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.order_purchase_date_key = d.date_key
GROUP BY HOUR(d.full_date)
ORDER BY total_orders DESC
LIMIT 5;
```

6. Dashboard / ML Model

6.1 Power BI Dashboard



1. Overview

This section documents the **Sales & Customer Dashboard** created in Power BI, presenting key business metrics, pricing analysis, customer RFM segmentation, revenue trends over time, and customer geographic distribution.

2. KPI Cards

- **Total Revenue | \$15.74M**
 - Cumulative payment value across all orders.
- **Total Customer | 96,092**
 - Total count of distinct customers.
- **Avg Product Price | \$145.7**

- Average price of products.
- **Avg Order Freight | \$19.99**
 - Average freight value per order.

3. Dashboard Visuals

A. Average Product Price by Category (Bar Chart)

Displays the top categories ranked by average product price:

- **pcs: \$1290**
- **portateis_casa_forno_e_cafe: \$523**
- **eletrodomesticos_2: \$450**
- **agro_industria_e_comercio: \$410**
- **instrumentos_musicais: \$323**

B. Customers by RFM Segment (Donut Chart)

Segments the total customer base of **96.1K customers** into RFM categories:

- **Potential Loyalists: 29.0K (30.1%)**
- **Champions: 26.6K (27.7%)**
- **Frequent / Good Buyers: 17.5K (18.2%)**
- **Loyal Customers: 11.7K (12.2%)**
- **Low Value / Regular: 11.3K (11.8%)**

C. Sales Trend over Time (Area Chart)

Tracks revenue values across quarters:

- **2016-Q3:** 0.0M
- **2016-Q4:** 0.1M
- **2017-Q1:** 0.9M
- **2017-Q2:** 1.5M
- **2017-Q3:** 2.0M
- **2017-Q4:** 2.8M
- **2018-Q1:** 3.2M
- **2018-Q2:** 3.3M
- **2018-Q3:** 2.1M

Data Note on 2018-Q3 Trend Drop

The drop observed in **2018-Q3 (\$2.1M)** does not indicate a decline in business sales performance. Instead, it is due to **data truncation (partial data collection)**, as the dataset records conclude mid-quarter in August 2018 and do not represent a full 3-month reporting period.

D. Top Customer by Locations (Bar Chart)

Shows customer count across top states:

- **SP:** 42K
- **RJ:** 13K

- **MG:** 11K
- **RS:** 5.5K
- **PR:** 5K

4. Dashboard Slicers

- **Year Slicer:** Filters data by year (2016, 2017, 2018).
- **Month Slicer:** Dropdown filter for months (All).
- **RFM Segment Slicer:** Checkbox filter for RFM segments (Champions, Frequent / Good Buyers, Low Value / Regular).
- **State Slicer:** Dropdown filter for states (All).

6.2 ML Model (Late Delivery Prediction)

To extract actionable business value beyond historical reporting, we engineered a machine learning pipeline using PySpark MLlib to predict delivery performance.

Objective The primary goal of the model is to predict whether an e-commerce order will arrive late, allowing the business to proactively manage supply chain bottlenecks and improve customer satisfaction.

Feature Engineering & Model Selection

We transformed categorical variables and vectorized numerical features, focusing on physical logistics (product weight, freight value) and geographical data (customer state).

During the modeling phase, we evaluated multiple classification algorithms. We initially tested an alternative model, but it failed to capture the variance in the data and yielded no meaningful predictive improvements. As a result, we selected the **Random Forest Classifier** as our final model. Random Forest was chosen for its robustness against overfitting, its ability to handle categorical variables natively, and its strong interpretability.

Key Findings The Random Forest model successfully identified the primary drivers of shipping delays. The feature importance metrics revealed that logistical constraints (specifically `total_weight` and `freight_value`) combined with geographical destinations (`customer_state`) were the highest predictors of late deliveries, rather than internal payment or order processing times. This insight enables the business to target specific shipping routes and heavy-freight carriers for supply chain optimization.

7. Conclusion

The project successfully implemented an end-to-end data engineering pipeline for processing and analyzing e-commerce data. The data was loaded from MariaDB into HDFS using a JDBC connection for distributed storage and processing.

Spark was used to perform data cleaning and transformation, including historical product price tracking, product and customer dimension development, RFM analysis, and customer segmentation. The transformed data was then organized into a Star Schema and implemented as Hive tables stored in Parquet format.

Hive queries were used to analyze the data warehouse and extract meaningful business insights. A Power BI dashboard was developed to visualize these insights, while a machine learning model was implemented to predict late deliveries.

Overall, the project integrates data ingestion, distributed data processing, data warehousing, business intelligence, and machine learning into a unified solution that transforms raw e-commerce data into actionable business information.

8. Future Work

Future improvements could include implementing incremental data loading to process newly arriving data more efficiently and reduce the need for full data reprocessing.

The existing machine learning model could also be improved by experimenting with additional features and machine learning algorithms to enhance late-delivery prediction.

Another potential extension is developing a customer churn prediction model to identify customers who are likely to stop purchasing. This could be combined with the existing RFM analysis and customer segmentation to support targeted customer retention strategies.

The Power BI dashboard could also be enhanced with additional KPIs, interactive visualizations, and more advanced customer and product analytics.

